

A new Ad-Hoc parallel file system for HPC environments based on the Expand parallel file system

1st Felix Garcia-Carballeira

Computer Science and Engineering Departament
Universidad Carlos III de Madrid
Leganes, Spain
felix.garcia@uc3m.es

2nd Diego Camarmas-Alonso

Computer Science and Engineering Departament
Universidad Carlos III de Madrid
Leganes, Spain
dcamarma@inf.uc3m.es

3rd Alejandro Caderon-Mateos

Computer Science and Engineering Departament
Universidad Carlos III de Madrid
Leganes, Spain
acaldero@inf.uc3m.es

4th Jesus Carretero

Computer Science and Engineering Departament
Universidad Carlos III de Madrid
Leganes, Spain
jcarrete@inf.uc3m.es

Abstract—An Ad-Hoc File System dynamically virtualizes storage on compute nodes into a fast storage volume to reduce congestion on parallel file systems used as backends in HPC environments and improve data locality. This paper presents Expand Ad-Hoc, a version of the Expand parallel file system, for use as an Ad-Hoc storage system for HPC environments. Such an update seeks to take better advantage of new storage technologies (on SSDs local to nodes, for example) and to adapt to new demands of current parallel applications (e.g., optimizing data access by analyzing data locality). The paper describes the new system's features and presents a first evaluation comparing the performance of Expand with another Ad-Hoc File System (GekkoFS), and GPFS. The first results are quite satisfactory and demonstrate the good performance of Expand as an Ad-Hoc storage system.

Index Terms—Ad-Hoc File System, HPC, Parallel I/O

I. INTRODUCTION

Currently, there are different parallel and distributed file systems [1] [2]. However, all of them need to adapt to the needs of new applications, storage devices, and communications, among others.

For example, in recent years, new areas of work have emerged, such as Artificial Intelligence or Big Data, where applications usually perform massive data analysis through the execution of workflows. Being the workloads and data access patterns different from traditional High-Performance Computing (HPC) applications [3] [4]. Therefore, it is necessary that the file system used to store the data is designed for this type of application and offers the required performance.

As an additional example, another problem is that the share storage space available on nodes is usually limited. Still, a scratch space local to nodes can be used during the execution so that the aggregate space of all the scratch nodes allows a larger (and usually faster) storage space than the shared space.

As for technological improvements, they are usually implemented internally in the operating system by hiding the changes under existing interfaces, such as sockets for communications or under the POSIX API [5] for files and directories. However, sometimes it is impossible to hide these changes under an existing interface, and it is necessary to adapt the file system architecture to make use of these improvements and obtain maximum performance.

This paper introduces the Expand Ad-Hoc File System, a distributed and parallel file system designed and improved to adapt to new application needs and existing technologies. The novel aspects of this work are the following:

- A new Ad-Hoc parallel file system based on MPI to obtain a portable file system on multiple platforms.
- New locality mechanisms to enhance the I/O performance.
- Capacity to use POSIX applications without modifications through intercepting system calls.
- Scalability and good performance for parallel applications.

The rest of the paper is structured as follows: Section II presents the State of the Art; Section III describes the design of the Expand Ad-Hoc architecture; Section IV shows the results of the performance offered by Expand Ad-Hoc. Finally, Section V presents the main Conclusions and Future Work.

II. RELATED WORK

A parallel file system is a kind of distributed file system that distributes file information among multiple servers and provides parallel access [2].

The following are some of the major parallel file systems for HPC:

- GPFS [6] began as the Tiger Shark file system, a research project at IBM's Almaden Research Center as early as 1993. Tiger Shark was initially designed to support high-performance multimedia applications. This design turned out to be well suited for scientific computing.
- Lustre [7] was started as a research project in 1999 by Peter J. Braam, who at the time was on staff at Carnegie Mellon University (CMU). Braam went on to found his own company Cluster File Systems in 2001, building on work on the InterMezzo file system [8] in CMU's Coda project. Lustre is generally used for large-scale cluster computing. The name Lustre is a word derived from Linux and cluster.
- Parallel Virtual File System (PVFS) [4] is designed for use in large-scale clusters and as an open-source project in 2002. PVFS focuses on high-performance access to large data sets. It consists of a server process and a client library, both written entirely with user-level code. The client library enables high-performance access through the Message Passing Interface (MPI) [9]. PVFS is now known as OrangeFS [10] in its most recent development branch.
- Expand 1.0 [11] was created in 2003 with the main motivation of building a parallel file system for general purpose heterogeneous distributed environments. Expand uses the existing Network File System (NFS) [12] to satisfy this goal. No changes to the NFS servers are required. The NFS servers use the cache internally. All operations are implemented on the clients.
- GekkoFS [13] is a file system bootstrapped in 2017 that aims to leverage the performance advantage of using node-local NVRAM disk drives to accelerate I/O for scientific applications. The basic design idea of GekkoFS is to build a distributed storage space from node-local storage devices with the same ephemeral lifecycle of a batch-submitted job. By aggressively distributing data and metadata among cluster nodes, GekkoFS efficiently and scalably aggregates the I/O performance of node-local storage.

As you can see, the evolution over the years has allowed it to adapt to the new needs of today's applications and new storage and communication technologies.

A. Ad-Hoc File Systems

The increasing size of HPC systems increases the likelihood of congestion in parallel file systems used as a backend in HPC environments. An Ad-Hoc File System dynamically virtualizes storage on compute nodes into an application-tailored, fast storage volume to reduce congestion in parallel backend file systems and improve data locality.

An Ad-Hoc File System is a simple way to implement efficient storage hierarchies in HPC environments. They can be deployed on the local storage of compute nodes to build storage systems that provide faster I/O access to applications. And it also allows adjusting the storage to be used by the application to its needs or the resources available at the time

of execution using the aggregated local storage (being able to offer more or less storage space according to the need of the application in a given execution and the available nodes).

Ad-Hoc storage systems can be classified into short-lived Ad-Hoc storage systems deployed in the context of an application run and long-lived Ad-Hoc storage systems that support long-lived workflows. An Ad-Hoc storage system's lifetime is much shorter than that of a parallel file system, so reliability constraints are easier to meet. However, it is necessary to provide malleability features to support long-lived workflows or long simulations and adapt to changing application demands, e.g., after the number of nodes is changed for a malleable application.

An early file system optimized to be deployed as a local file system was the BurstFS ephemeral file system [14]. The main drawback of this approach is that these file systems have to bear the entire burden of the original implementation, which includes deployment and initialization of metadata servers, redundancy mechanisms that are unnecessary for short-lived file systems, and full POSIX compliance. This approach leads to long startup times, which can easily be on the order of the lifetime of the jobs using these file systems. Other examples of Ad-Hoc File Systems are GekkoFS [15], and UnifyFS [16].

III. EXPAND AD-HOC

Expand Ad-Hoc ¹ is a parallel file system based on user-space Ad-Hoc servers that have been designed for use in HPC environments where data-intensive applications are running. The following will describe its design and the main features that enable its use in the above environments.

A. Design of Expand Ad-Hoc File System

The Expand Ad-Hoc design is based on the original Expand [11] design which is a parallel, distributed file system based on the use of existing standard storage servers (e.g., NFS). However, the original architecture has had to be modified to make it an Ad-Hoc parallel file system.

The following figure (Figure 1) shows the result of the Expand architecture redesign to convert it into an Ad-Hoc Parallel File System.

This new architecture is based on a set of data servers running on different compute nodes that communicate with each other using MPI. The use of MPI facilitates the standardization of Expand and its use in HPC environments.

For data storage, parallel data partitions are created on the local storage devices (HDD, SSD, or RAM) of the various Ad-Hoc servers deployed. Therefore, the services provided by the local operating system (e.g., POSIX) are used to manage this data.

The Ad-Hoc applications and clients are deployed on the compute nodes and communicate with the Expand Ad-Hoc servers using MPI, just like the data servers.

As shown in Figure 1, Ad-Hoc applications and clients don't have to be deployed on compute nodes that also run Ad-Hoc

¹<https://xpn-arcoos.github.io/>

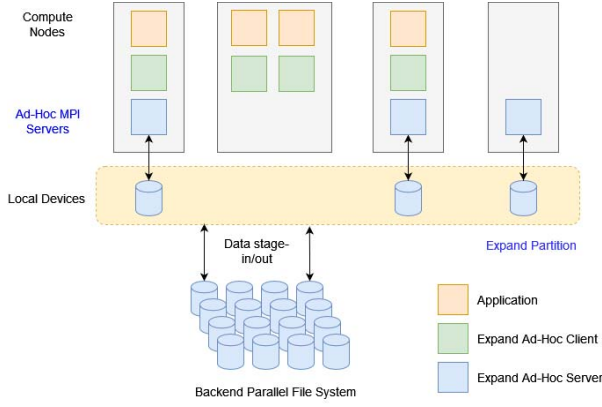


Fig. 1. Architecture of the Expand Ad-Hoc File System.

servers. The same is true for Ad-Hoc servers. They can be deployed on compute nodes that do not run applications.

Figure 2 describes the internal details of Expand Ad-Hoc as an Ad-Hoc parallel file system, and the following sections will go into more detail on each of the parts.

B. Data Distribution

Expand Ad-Hoc combines several Expand Ad-Hoc servers using MPI (see Figure 2) to provide a generic parallel partition. For this purpose, each Ad-Hoc server offers one or more directories that are combined to build the distributed partition that will be used on the compute nodes.

The files stored in the file system will be divided into blocks among all the Ad-Hoc servers deployed to facilitate parallel access. Each server will be conceptually holding a subfile of the original file. For this reason, a file stored in Expand Ad-Hoc consists of several subfiles, one for each Ad-Hoc server. In addition, all subfiles created are entirely transparent for Expand Ad-Hoc users.

The blocks are distributed among the subfiles (in the parallel partition) following a cyclic or it round-robin pattern, as seen in Figure 3. This way, the blocks are distributed equally among all the deployed Ad-Hoc servers.

Figure 3 shows that the user data file is divided into different blocks distributed among the Ad-Hoc servers. On the other hand, the Expand Ad-Hoc block size is used to distribute the blocks among the servers, being this size independent of the one used in the parallel file system of the backend.

C. Metadata Management

To define the different Expand Ad-Hoc partitions is necessary to create a configuration file in XML format, as shown in the Listing 1. This configuration file for each partition defines its name, the Expand block size (partitioning size), and the nodes that make up this partition, among other parameters.

For example, in the configuration file shown in the Listing 1, a partition ("p1") with two Ad-Hoc servers with a block size of 64 KiB is defined. In addition, for each server, the directory where each server's subfiles will be stored is specified.

Listing 1. Configuration file used to define an Expand partition

```
1 <?xml version="1.0" encoding="ISO-8859-1"?>
2 <xpn_conf>
3   <partition name="p1" bsize="64k">
4     <data_node
5       id="<node ID>"
6       url="mpi_server://<server>/<path1>"/>
7     <data_node
8       id="<node ID>"
9       url="mpi_server://<server>/<path2>"/>
10   </partition>
11 </xpn_conf>
```

Expand uses two levels of metadata management. A first level in the master node that only stores the distribution policy, block size, and the node that stores the first block. The rest of the metadata (size, owner, etc.) is managed by the local file system of each node and is therefore distributed by all nodes. Therefore, the master node is not a bottleneck because metadata management is distributed among all nodes.

The associated metadata of the files stored in Expand Ad-Hoc is preserved in a small header at the beginning of each subfiles (see Figure 3). The metadata used by Expand Ad-Hoc includes the block size (cast size), base node (identifies the Ad-Hoc server where the first block of the file resides), and the file distribution pattern used. At the moment, only cyclic or "round-robin" distribution is available.

However, although all subfiles have a header to store the metadata, only one of the partition nodes, called the master node (which may be different from the base node), will store the file metadata.

In addition, to simplify the file naming process and reduce potential bottlenecks, Expand Ad-Hoc does not use any kind of metadata manager. Figure 3 shows how Expand Ad-Hoc performs directory mapping, and stores file metadata in the header of one of the subfiles stored on the Ad-Hoc servers.

The file name is converted to a node number by applying a hash function to identify the master node.

Therefore, as the assignment of the master node depends on the file name, when this name is modified by a user the master node must also change. So the algorithm used in Expand Ad-Hoc to rename a file is as follows:

Listing 2. Algorithm to rename a file

```
1 xpn_rename(old-name, new-name) {
2   old-master = hash(old-name)
3   new-master = hash(new-name)
4   move-metadata(old-master, new-master)
5   storage_rename(old-name, new-name)
6 }
```

D. Parallel Access

Expand Ad-Hoc uses a virtual file handler to reference all operations performed on files. Therefore, when a subfile needs to be accessed, the appropriate file handler is used, allowing I/O to be optimized as the Expand Ad-Hoc library splits user requests into distinct parallel sub-requests sent to the servers involved in the operation.

That is, a request involving k Ad-Hoc servers, Expand Ad-Hoc performs k requests in parallel to the servers using threads to parallelize these operations. This criterion applies to all operations that can be performed in Expand Ad-Hoc.

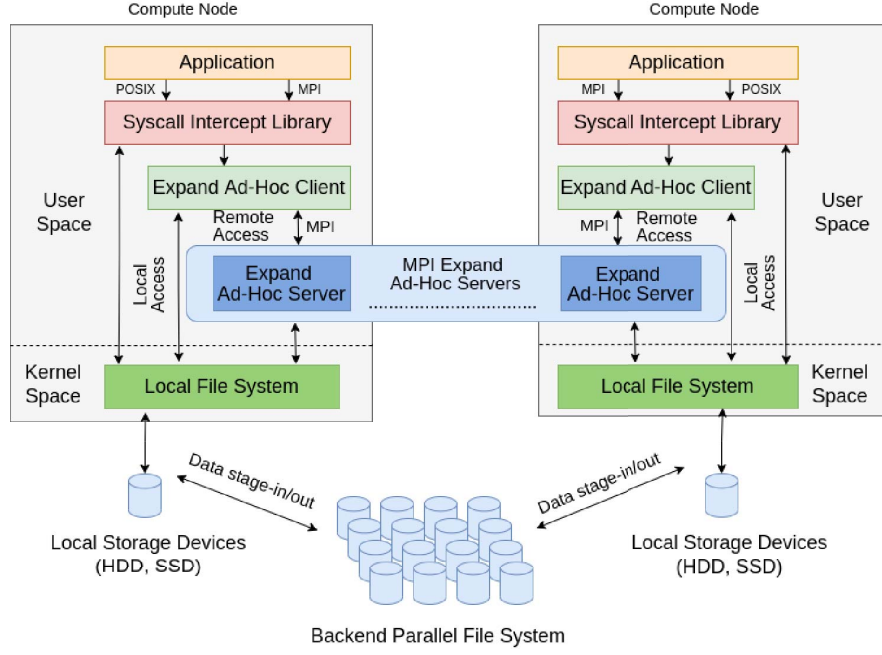


Fig. 2. Expand Ad-Hoc architecture details.

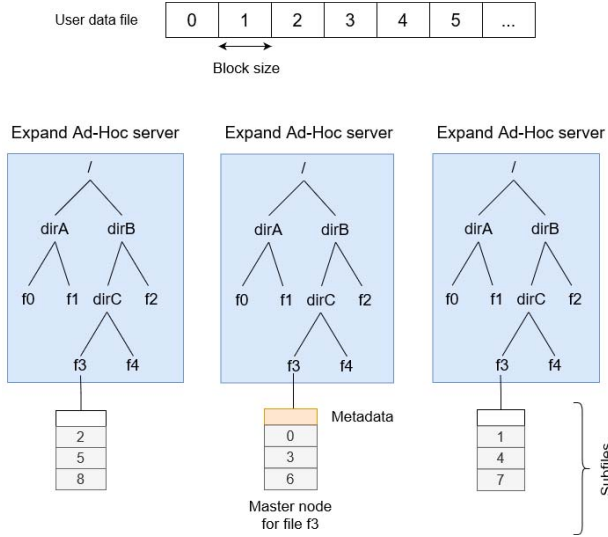


Fig. 3. File layout and directory mapping in Expand.

E. Data Locality

As explained in Section III-A, Expand Ad-Hoc servers can be deployed on the same compute nodes on which the parallel application is running. Thus, that application can use Expand Ad-Hoc to store its data. When an application process running on one node accesses the data, this data could be in the same node or another remote node.

When the first case occurs, Expand Ad-Hoc can detect and

take advantage of the data locality to access the data directly using the local file system (e.g., a POSIX file system) without contacting the server that manages the data. This behavior allows optimizing data access from the Expand Ad-Hoc client since the overhead and latencies caused by remote access (e.g., using MPI) in data access are avoided.

Note that data locality is very important when the same data set must be processed by a sequence of parallel applications (workflow) running on the same set of nodes. The typical scenario for these applications involves the following operations:

- 1) Optionally, the initial data can be copied from a permanent storage space (e.g., Lustre, CEPH, etc.) to the Expand Ad-Hoc server storage.
- 2) The first application in the workflow saves the data to the Expand Ad-Hoc servers, which is processed by the application deployed on these same nodes.
- 3) The different parallel applications that are executed afterward continue working with the data and taking advantage of the locality provided by these nodes.
- 4) Optionally, the final results can be copied to a permanent storage space (e.g., Lustre, CEPH, etc.).

F. I/O Data Staging

As noted in Section III-D, data input operations are performed in parallel from the backend parallel file system (e.g., Lustre) to the Ad-Hoc servers without the need for client applications to intervene (see Figure 2). Thus, each Ad-Hoc server generates a subfile in its local storage space by reading the data from the file stored in the backend file system.

Similarly, data output is also performed in parallel. In this case, each Ad-Hoc server writes the data blocks stored in its local subfile to the final file of the parallel file system backend.

Although these operations could be performed using a POSIX application, two new system calls have been developed in Expand Ad-Hoc that allow these operations to be performed directly from the Ad-Hoc servers, improving performance. These calls are:

- Preload: allows copying a file from the backend file system to an Expand Ad-Hoc partition.
- Flush: writes a file from an Expand Ad-Hoc partition to the backend file system.

G. System Call Interception

Expand Ad-Hoc has a system call interception library that makes it unnecessary to modify the source code of existing or legacy applications to use this file system.

This library allows intercepting the vast majority of POSIX system calls made during the execution of a program. The corresponding API function is called if the file involved is stored in Expand Ad-Hoc. However, if the system call is made on the local file system of the compute node, the corresponding system call will be executed in the `libc.so` (see Figure 2).

`LD_PRELOAD` is used to load the intercept library before the rest (including the Linux `libc.so`) without the need for super-user permissions. There are other technologies, such as FUSE [17], to perform this same operation, but it uses a block size of 128KiB, limiting to this value the block size of the Ad-Hoc File System. It also requires super-user permissions as it is in the Linux kernel, which makes it very difficult to use and to use in HPC environments.

However, it is also possible to use the Expand Ad-Hoc API functions to interact with the files in this file system. But to use the Expand Ad-Hoc API, the source code must be modified, and the application must be compiled with the Expand Ad-Hoc dynamic library. These functions are very similar to the POSIX API calls both in the name (e.g., `xpn_open`, `xpn_read`, etc.) and the arguments they receive (e.g., `xpn_read(fd, buf, nbytes)`).

IV. EVALUATION

The main goal of this section is to evaluate the I/O performance of Expand Ad-Hoc when a parallel application writes and reads data on a shared file. The evaluation compares the performance of GPFS, GekkoFS, and Expand Ad-Hoc.

The IOR benchmark² has been chosen to perform the evaluation. IOR is an open source benchmark that is commonly used to evaluate the performance of distributed and parallel file systems by simulating different I/O loads (read/write operations, single/shared file access, etc.).

We have used a standard benchmark (IOR) with an access pattern (file sharing) that is challenging for most parallel file systems. Expand delivers good results because MPI is

optimized for network usage and because Expand's locality strategies allow to reduce the load on servers and network.

The evaluation has been performed in Marenostum IV. Marenostum IV [18] has 3456 Lenovo ThinkSystem SD530 compute nodes on 48 racks. In total, 165888 processes and 384.75 TiB of main memory.

Each node:

- Uses two Intel Xeon Pentium 8160 24C chips with 24 processors, each at 2.1 GHz.
- Provides an Intel SSD DC S3520 Series with 240 GiB of available storage, usable within a compute job.
- Can access up to 96 GiB (2 GiB per core) of main memory.
- Has a 100 Gb Intel Omni-Path Full-Fat Tree is used for the interconnection network connected to a total of 14 PiB storage capacity by IBM's GPFS.

In the evaluation, the default configuration has been used for all file systems:

- Storage:
 - Expand Ad-Hoc and GekkoFS are configured to use a block size in both cases of 512 KiB to compare both file systems in the same conditions.
 - Both Expand Ad-Hoc and GekkoFS use the local SSD and the shared memory available (SHM) as underlying storage. For SHM both systems use `/dev/shm` as underlying file system.
- Network:
 - GekkoFS was used with the default stable configuration available.
 - In Expand Ad-Hoc, the Intel MPI was used with default configuration (without any configuration parameters added).

To evaluate Expand Ad-Hoc, different performance tests have been executed to obtain the throughput of write and read in parallel accesses on the same file. In addition, the performance tests performed are based on [13] to evaluate GekkoFS

The following configurations have been used for testing with IOR:

- Compute nodes: 1, 2, 4, ... up to 128 compute nodes.
- Local storage: SSD and Shared Memory (SHM).
- Transfer size used in IOR: 64 KiB, 512 KiB, and 1 MiB.
- Client processes per compute node: 8.
- Operations: read and write in parallel on a shared file.
- Size written by each client: 4 GiB (resulting in a 4 TiB of shared file in the maximum configuration).
- File systems: GPFS, GekkoFS 0.9.1, and Expand Ad-Hoc 2.0.

In all tests, one Expand Ad-Hoc server was used for each compute node (1 up to 128), and the same server configuration was in GekkoFS. In addition, each of these tests have been run as individual jobs with the compute nodes in exclusivity using SLURM (Simple Linux Utility for Resource Management) [19].

²<https://github.com/hpc/ior>

All tests have been executed 10 times and the average results and standard deviation are shown in all figures.

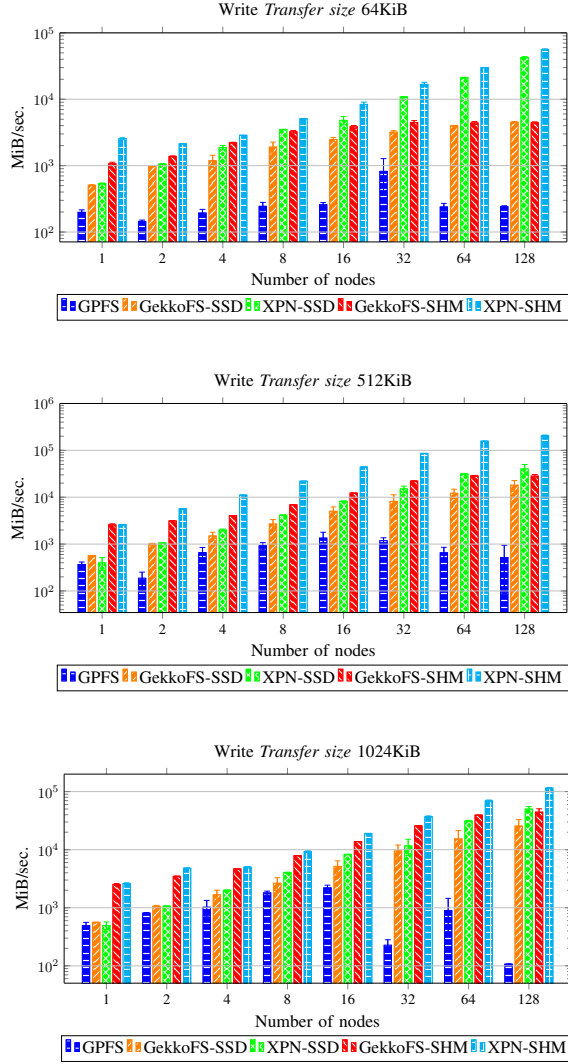


Fig. 4. GPFS vs. GekkoFS vs. Expand Ad-Hoc. Throughput (MiB/sec.) writing with different transfer sizes (64KiB, 512KiB, and 1MiB), compute nodes (1, 2, 4, 8, 16, 32, 64, and 128), with 8 client process per node and shared file. Results in logarithmic scale.

In Figures 4 and 5 the X axis shows the different number of nodes used (1, 2, ... 128 nodes) and the Y axis the throughput obtained in MiB/sec. with logarithmic scale. Specifically, Figure 4 shows the throughput for writing with transfer sizes of 64 KiB, 512 KiB, and 1024 KiB. In Figure 5 the throughput for reading is also shown with 64 KiB, 512 KiB, and 1024 KiB transfer size. Each Figure shows the results of GPFS, GekkoFS with SSD (GekkoFS - SSD) and with share memory (GekkoFS - SHM), and Expand Ad-Hoc with SSD (XPN Ad-Hoc - SSD) and with share memory (XPN Ad-Hoc - SHM).

If we analyze the GPFS file system, we can see that for write operations, as the number of nodes increases, the

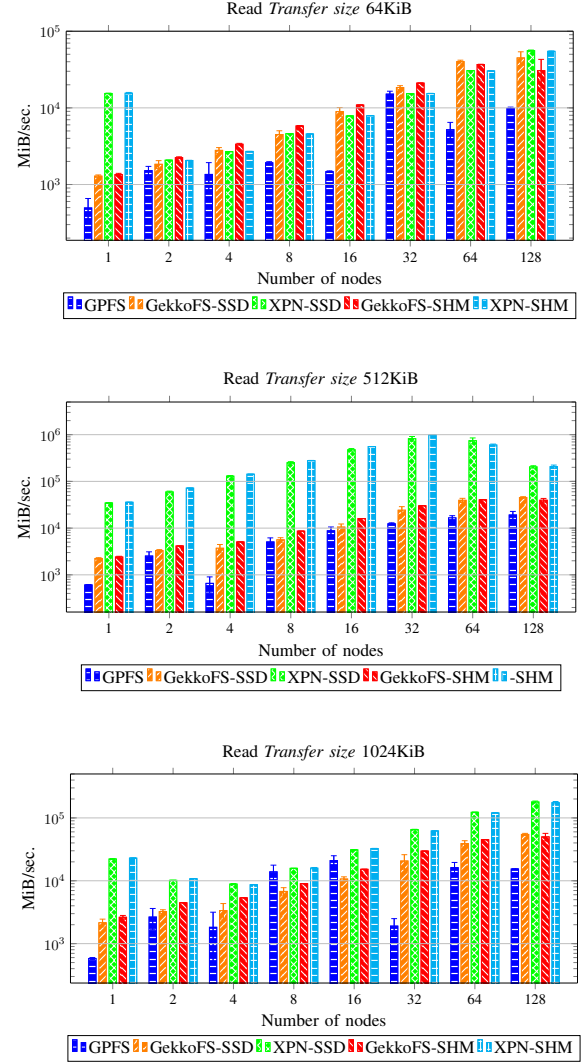


Fig. 5. GPFS vs. Expand Ad-Hoc. Throughput (MiB/sec.) reading data with different transfer sizes (64KiB, 512KiB, and 1MiB), compute nodes (1, 2, 4, 8, 16, 32, 64, and 128), with 8 client process per node and shared file. Results in logarithmic scale.

throughput decreases. This is because the number of GPFS storage nodes is fixed, so it does not help to scale with the number of clients (which is the case with most non-Ad-Hoc file systems). Regarding GPFS reads, we can see that in general the throughput does increase as the number of nodes increases. This behavior is due to the fact that GPFS uses a cache on the nodes where the clients are running, so it takes advantage of the locality of the data to increase its performance with respect to writes.

If we analyze the performance offered by the GekkoFS file system in writes we can say that when using a transfer size of 64 KiB the throughput from 16 upwards is similar. This is due to the fact that the transfer size used is lower than the GekkoFS block size which stabilizes the performance.

However, when the transfer size is 512 KiB and 1024 KiB the throughput keeps increasing with the number of nodes as it is multiple of the default GekkoFS block size. As for the readings for all transfer sizes, the behavior is similar, the throughput increases as the number of nodes increases, but from 64 nodes upwards the performance stabilizes. To conclude the analysis of the GekkoFS results, we can observe that the behavior when using shared memory as local storage is very similar to that obtained when using SSD as storage. However, the throughput is higher since the throughput offered by shared memory (based on RAM) is higher than that offered by SSD. This throughput difference is very significant when few nodes are used, but becomes smaller as the number of nodes increases (this is because there is more network usage, which generates a bottleneck).

Finally, the Expand Ad-Hoc file system will be analyzed. Regarding the performance obtained in the writes, it can be seen that the throughput scales as the number of nodes increases. And the performance is similar among the different transfer sizes. As for reads, in general the throughput increases as the number of nodes increases, as was the case with writes. However, there are two points to highlight. First, when only one node is used, the throughput is much higher than when several nodes are used because there is total locality of the data and no network is used (Section III-E). Second, when a transfer size of 512 KiB is used, coinciding with the Expand Ad-Hoc block size, there is total locality regardless of the number of nodes (due to the distribution of blocks made by IOR). This means that from 64 nodes upwards, performance drops as the local storage used becomes saturated. If we analyze the results obtained when Expand Ad-Hoc uses shared memory, we can say that in both writes and reads the performance is similar to that obtained using SSD. The difference in throughput between SSD and shared memory is especially noticeable in the writes, since in the reads, although SSD is used, the OS cache comes into play, equalizing the performance.

Finally, we can compare the performance seen in the Figures. If we analyze the performance of GPFS with respect to the rest, we see two main limitations: scalability, especially in write operations, which is limited when increasing the number of nodes. As a second limitation, the throughput offered is much lower than the rest, especially when there are many clients. Comparing the two Ad-Hoc file systems analyzed in this work (GekkoFS and XPN Ad-Hoc) we can see that the throughput offered by XPN Ad-Hoc is higher than the GekkoFS throughput in most of the tests for both writes and reads. And using data locality greatly helps to increase performance, especially in read operations.

V. CONCLUSIONS AND FUTURE WORK

This paper has described the Expand Ad-Hoc parallel file system for HPC environments and the evaluation performed in order to compare with GPFS and GekkoFS (another well-known Ad-Hoc parallel file system). The Expand Ad-Hoc is

available as an open-source project in the following GitHub repository: <https://github.com/xpn-arcos/xpn>

Expand Ad-Hoc adds to the benefits of Expand and the possibility of using user-space MPI-based data servers on the same compute nodes on which applications are running. Moreover, Expand Ad-Hoc can be used with HDD/SSD local storage and share memory (SHM through `/dev/shm`) as well. Allowing it to be used in systems that do not have local storage in the nodes, but do have shared memory.

As main conclusions from the evaluation results, Expand Ad-Hoc provides the expected performance, simplifying the implementation of a parallel file system and allowing to take advantage of both worker nodes and specific storage nodes.

Finally, we are working on several main lines for future work. One of these lines of work consists of the design and implementation of malleability techniques and fault tolerance in this new file system. In addition, work is also being done to improve the use of file metadata in Expand Ad-Hoc.

It is also planned to analyze the behavior of Expand Ad-Hoc to store part of the information used by the image analysis tools used in the ERA4TB project.

At last but not least, we are planning the evaluation of Expand Ad-Hoc with the 10 nodes IO500 challenge. Right now Expand Ad-Hoc can execute IO500 without any issues, but the full evaluation is one of our future work.

ACKNOWLEDGMENT

This work has been partially funded by the Spanish Ministry of Science and Innovation project “Expand: High Performance Storage System for HPC and Big Data Environments”. Ref. TED2021-131798B-I00.

And partially funded by the “Adaptive multi-tier intelligent data manager for Exascale (ADMIRE)” project of the European Union’s Horizon 2020 JTI-EuroHPC research and innovation programme. Ref. H2020-JTI-EuroHPC-2019-1, Project no. 956748.

And partially supported by the EuroHPC project “Adaptive multi-tier intelligent data manager for Exascale” under grant 956748 — ADMIRE — H2020-JTI-EuroHPC-2019-1 and by the Agencia Española de Investigación under Grant PCI2021-121966.

The author thankfully acknowledges the computer resources at MareNostrum and the technical support provided by Barcelona Supercomputing Center (IM-2022-3-0009).

REFERENCES

- [1] P. Shah, J. Ye, and X.-H. Sun, “Survey the storage systems used in hpc and bda ecosystems,” 2021. [Online]. Available: <https://arxiv.org/abs/2112.12142>
- [2] V. Dubeyko, “Comparative analysis of distributed and parallel file systems’ internal techniques,” 2019. [Online]. Available: <https://arxiv.org/abs/1904.03997>
- [3] A. J. Hey, S. Tansley, K. M. Tolle *et al.*, *The fourth paradigm: data-intensive scientific discovery*. Microsoft research Redmond, WA, 2009, vol. 1.
- [4] R. B. Ross and R. Latham, “PVFS - PVFS: a parallel file system,” in *SC*. ACM Press, 2006, p. 34.

- [5] I.-S. S. Board, "IEEE standard for information technology-portable operating system interface (POSIX)-part 1: System application program interface (API)- amendment D: Additional Real time Extensions [C Language]," National Institute of Standards and Technology, Tech. Rep., 1999.
- [6] F. B. Schmuck and R. L. Haskin, "GPFS: A shared-disk file system for large computing clusters," in *FAST*, D. D. E. Long, Ed. USENIX, 2002, pp. 231–244.
- [7] P. Braam, "The lustre storage architecture," 2019. [Online]. Available: <https://arxiv.org/abs/1903.01955>
- [8] P. Braam, M. Callahan, P. Schwan *et al.*, "The intermezzo file system," in *Proceedings of the 3rd of the Perl Conference, O'Reilly Open Source Convention*, 1999.
- [9] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard Version 4.0*, Jun. 2021, accessed Jan. 2, 2023. [Online]. [Online]. Available: <https://www.mpi-forum.org/docs/mpi-4.0/mpi40-report.pdf>
- [10] M. M. D. Bonnie, B. Marshall, W. B. Ligon, N. Mills, E. Q. S. Sampson, S. Yang, and B. Wilson, "OrangeFS: Advancing PVFS," 2011.
- [11] F. G. Carballeira, A. Calderón, J. Cartero, J. Fernández, and J. M. Pérez, "The design of the Expand Parallel File System," *Int. J. High Perform. Comput. Appl.*, vol. 17, no. 1, pp. 21–37, 2003.
- [12] D. B. Noveck and C. Lever, "Network file system (NFS) version 4 minor version 1 protocol." *RFC*, vol. 8881, pp. 1–560, August 2020. [Online]. Available: <http://dblp.uni-trier.de/db/journals/rfc/rfc8800-8899.html>
- [13] M. Vef, N. Moti, T. Süß, M. Tacke, T. Tocci, R. Nou, A. Miranda, T. Cortes, and A. Brinkmann, "GekkoFS - A temporary burst buffer file system for HPC applications," *J. Comput. Sci. Technol.*, vol. 35, no. 1, pp. 72–91, 2020.
- [14] T. Wang, K. Mohror, A. Moody, K. Sato, and W. Yu, "An ephemeral burst-buffer file system for scientific applications," in *SC'16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2016, pp. 807–818.
- [15] M.-A. Vef, N. Moti, T. Süß, T. Tocci, R. Nou, A. Miranda, T. Cortes, and A. Brinkmann, "GekkoFS: a temporary distributed file system for HPC applications," in *2018 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2018, pp. 319–324.
- [16] A. Moody, D. Sikich, N. Bass, M. J. Brim, C. Stanavice, H. Sim, J. Moore, T. Hutter, S. Boehm, K. Mohror *et al.*, "Unifyfs: A distributed burst buffer file system-0.1.0," Lawrence Livermore National Lab.(LLNL), Livermore, CA (United States), Tech. Rep., 2017.
- [17] B. K. R. Vangoor, P. Agarwal, M. Mathew, A. Ramachandran, S. Sivaraman, V. Tarasov, and E. Zadok, "Performance and resource utilization of FUSE user-space file systems," *ACM Trans. Storage*, vol. 15, no. 2, may 2019. [Online]. Available: <https://doi.org/10.1145/3310148>
- [18] BSC, "MareNostrum specification," 2023, accessed Jan. 2, 2023. [Online]. [Online]. Available: <https://www.bsc.es/marenostrum/marenostrum/technical-information>
- [19] SLURM, "Slurm workload manager," 2023, accessed Jan. 2, 2023. [Online]. [Online]. Available: <https://slurm.schedmd.com/documentation.html>